DeployGuard

Module-by-module audit and viva preparation

What this document contains

A guided review of 17 product modules: each claim, how it works, where to demonstrate it, likely failure cases, and its stated verification status. It also includes examiner questions, a five-minute presentation script, and a demo checklist.

How to read the status labels

PASS means a structural or local check was reported as passing in the supplied audit. **UNVERIFIED** means the specific live AWS or external behavior was not demonstrated in that audit. **DIFFERENT FROM PDF** identifies a mismatch between the module document and the inspected implementation.

Evidence boundary

This PDF formats the supplied audit. Its test results and code references are reported findings from that text; this conversion did not run the repository or independently certify AWS behavior.

Source snapshot: test2 at a571c14e9afc45bedf22ec1e4d22e136e8e3005e; the supplied audit notes a dirty worktree and a separate test1 reference.

Repository snapshot

test2 at a571c14e9afc45bedf22ec1e4d22e136e8e3005e.

Important: the repository instructions specify test1, but the working tree is on test2. test1 currently points to 6dda9a7fc2c605885e0426acc6ac54fa8c5cc590. The worktree is also dirty. My audit is against the checked-out code, not uncommitted changes as certified release proof.

The PDF is the module document, not an executable instruction. I treated its 17 descriptions as claims and compared them with the code.

Module-by-module viva audit

1. User Authentication & Audit Logging

Claim: secure login, roles, sessions, and audit records.

- Meaning: users must authenticate; different roles have different permissions; important actions are recorded.
- Code: auth.controller.ts, auth.service.ts, require-role.guard.ts, audit-log.service.ts.
- How: GitHub OAuth validates state, creates an HTTP-only session cookie, middleware loads the user, and role guards protect APIs. Audit records store actor, action, resource, status, IP, and sanitized metadata.
- Live demo: log in, open a protected project page, attempt an admin-only action as a developer, then show the audit log.
- Why: OAuth avoids storing GitHub passwords; server-side sessions and RBAC centralize security.
- Failure: invalid OAuth state, disabled account, expired session, missing session secret, or incorrect role.
- Status: **PASS** structurally. Authentication verification passed locally.

Likely question: “Why use an HTTP-only cookie?”

Answer: “JavaScript cannot read it, so it reduces token theft through client-side scripts.”

2. Project & Workspace Management

Claim: users create projects, connect GitHub, select directories, and view status/history.

- Code: projects.controller.ts, projects.service.ts, github-app.service.ts, frontend/src/pages/Projects.jsx, NewProject.jsx, and ProjectDetails.jsx.
- How: project data stores repository, branch, environment, services, variables, deployment runs, and current state.
- Live demo: create a project, connect GitHub, choose a branch and directory, then open Overview and Pipeline.
- Why: project configuration is the canonical input for repeatable deployments.
- Failure: inaccessible repository, invalid branch, missing directory, archived project, or unauthorized user.
- Status: **PASS** locally. GitHub/AWS access is **UNVERIFIED** without a live external run.

Question: “Why store deployment history?”

Answer: “It gives us traceability, retry decisions, stable releases, and rollback targets.”

3. Rule-Based Stack Detection Engine

Claim: repository files identify application structure, runtime, port, and deployment details.

- Code: build-target-resolver.service.ts, service-port-resolution, build-target-resolution, and deployable-service code.
- How: the resolver examines manifests, lockfiles, workspace files, service directories, and package managers. Port resolution uses repository evidence and canonical runtime configuration.
- Live demo: select a repository with a workspace and show the resolved service directory, build target, package manager, and port.
- Why: rule-based detection is deterministic and auditable; it avoids guessing from application behavior.
- Failure: unsupported language/framework, ambiguous workspace, missing start command, or conflicting port evidence.
- Status: **PASS**. Build-target verification passed.

Question: “Why not use AI for stack detection?”

Answer: “Deployment decisions need repeatable evidence. Rules are easier to test and explain.”

4. DevOps Template Engine & Pre-Flight Validation

Claim: requirements, commands, files, directory, and port are checked before deployment.

- Code: deployment-requirement-resolver.service.ts, docker-build.service.ts, and .github/workflows/deployguard-reusable.yml.
- How: DeployGuard resolves a canonical deployment contract, validates required inputs and secrets, builds the immutable image, and checks startup/port reachability before AWS mutation.
- Live demo: remove a required variable or give an invalid database URL and show admission blocking.
- Why: fail before infrastructure changes when the failure is already knowable.
- Failure: missing runtime variable, invalid database scheme, image startup failure, wrong binding address, or unavailable Docker.
- Status: **PASS**. Deployment-requirement verification passed.

5. CI/CD Pipeline & Asynchronous Queuing

Claim: GitHub Actions performs background deployment and DeployGuard tracks it.

- Code: pipeline.queue.ts, github-actions.service.ts, railpack-deployment.service.ts, and .github/workflows/deployguard-reusable.yml.
- How: the API creates an operation, queues or dispatches work, and later reconciles workflow stages, artifacts, logs, and terminal evidence.
- Live demo: click Deploy, immediately show the operation ID and pipeline stages, then refresh until completion.
- Why: builds and Terraform exceed normal HTTP request timeouts.
- Failure: duplicate dispatch, GitHub API error, workflow failure, timeout, or missing artifact.
- Status: **PASS** structurally. Exact external workflow execution is **UNVERIFIED**.

Question: “Why asynchronous instead of waiting for the API response?”

Answer: “Cloud work is long-running, so the API records the operation and the UI follows its persisted state.”

6. Security Scanning and Policy Enforcement

Claim: dependencies and images are scanned; critical fixable issues can block deployment.

- Code: .github/workflows/deployguard-reusable.yml Trivy steps, verify-trivy-controls.ts, and security result entities/services.
- How: Trivy scans immutable image references, classifies findings, hashes evidence, and blocks only when enforcement is enabled and the policy condition is met.
- Live demo: show advisory mode, then enforced mode with a fixture containing a fixable critical dependency.
- Why: scanning the exact release image prevents “scanned one artifact, deployed another.”
- Failure: scanner unavailable, malformed result, critical fixable dependency, or registry failure.
- Status: **PASS**. Trivy control tests passed.

7. Predictive Cost Analysis

Claim: estimated AWS cost is generated, saved, and shown with deployment.

- Code: finops.service.ts, infracost.service.ts, cost entities, and frontend/src/pages/Billing.jsx / project cost panels.
- How: Terraform cost plans can be converted to Infracost estimates. If real integration is disabled, the service uses an explicit mock estimate and records that fact.
- Live demo: generate an estimate, show resource breakdown, warning/approval state, and saved history.
- Why: cost should be visible before expensive infrastructure is created.
- Failure: invalid Terraform plan, unavailable Infracost, cost over limit, or approval rejection.
- Status: **PASS**, but distinguish real Infracost from demo/mock mode.

Question: “Is the displayed cost always real AWS billing?”

Answer: “No. It is an estimate. In mock mode the UI records that it is a mock estimate.”

8. Infrastructure Provisioning & Service Discovery

Claim: Terraform creates AWS resources and the endpoint is recorded.

- Code: infrastructure/railpack-runtime, backend/terraform/modules, terraform.service.ts, and service-discovery.service.ts.
- How: Terraform creates ECS, ALB, networking, EFS, secrets, Cloud Map, and logging resources. DeployGuard persists service-discovery and public endpoint evidence.
- Live demo: open Infrastructure after deployment and show Terraform events, ECS service, ALB health, and endpoint.
- Why: Terraform is strong at declarative infrastructure ownership.
- Failure: plan failure, apply failure, IAM/network problem, unhealthy target, or DNS/public verification failure.
- Status: **PASS** structurally; exact AWS provisioning is **UNVERIFIED**.

9. Distributed State Management

Claim: deployment/infrastructure state is stored safely and Terraform state is remote.

- Code: backend/src/state-management, state-management.service.ts, terraform-state.service.ts, and Terraform S3 backend configuration.
- How: PostgreSQL stores operation and projection state; S3 stores Terraform state; lock and validation services protect updates; recovery can restore a previous state version.

- Live demo: open State, show current version, lock status, checksum, validation, and previous versions.
- Why: separate operational history from infrastructure state and avoid local-only state.
- Failure: stale lock, corrupted state, unavailable S3, checksum mismatch, or recovery without authorization.
- Status: **PASS** structurally. Live AWS state behavior is **UNVERIFIED**.

10. Persistent Storage & Backup Recovery

Claim: persistent storage and restoration are available.

- Code: storage.service.ts, backup.service.ts, efs.service.ts, and backend/terraform/modules/efs.
- How: EFS can be mounted into ECS services; AWS Backup resources and restore requests are represented; storage settings and backup events are persisted.
- Live demo: enable storage, show EFS configuration and backup status, then submit a restore request.
- Why: application data must survive redeployments and infrastructure replacement.
- Failure: EFS mount failure, missing recovery point, restore failure, or database identity mismatch.
- Status: **PASS** structurally; actual backup/restore proof is **UNVERIFIED**. The PostgreSQL backup script did not run because required PORTABILITY_SOURCE_DATABASE configuration was absent.

Question: “Does Reset and Deploy Fresh delete healthy data?”

Answer: “It should not. Healthy persistent data is preserved; reset is admitted through the database reconciliation contract.”

11. Zero-Downtime Container Orchestration & Event-Driven Rollbacks

Claim: ECS manages releases and a stable previous release can be restored.

- Code: orchestration.service.ts, ecs.service.ts, project-stable-release.entity.ts, rollback services, and verify-direct-ecs-redeploy.ts / verify-direct-ecs-rollback.ts.
- How: ECS performs rolling deployment. DeployGuard records immutable image/task-definition identity, verifies stability and ALB health, and can update ECS to a previous exact task definition.
- Live demo: deploy version 1, deploy version 2, show stable release history, then roll back to version 1.
- Why: rollback must use a known immutable release, not rebuild from current source.
- Failure: no valid previous release, task-definition mismatch, unhealthy target, incompatible database secret, or ECS API failure.
- Status: **PASS** locally. AWS zero-downtime behavior is **UNVERIFIED**.

12. Dual-Stack Monitoring & Observability

Claim: deployment status, logs, and monitoring are available in one platform.

- Code: backend/src/observability, observability.controller.ts, CloudWatch services, Prometheus services, and frontend/src/pages/ProjectMetrics.jsx.
- How: DeployGuard combines pipeline events, CloudWatch logs/metrics, ECS runtime identity, and Prometheus/Grafana presentation.
- Live demo: open Metrics, view deployment timeline, stream logs, inspect runtime health, and open Grafana if configured.
- Why: operators need one diagnostic view across deployment and runtime.

- Failure: missing AWS permissions, unavailable log group, stale runtime identity, or disconnected Prometheus/Grafana.
- Status: **UNVERIFIED**. Source exists, but verify-aws-runtime-monitoring.ts currently failed because its expected UI text does not match the current component.

13. AI-Powered Log Analysis & Chatbot

Claim: logs are analyzed and troubleshooting guidance is provided.

- Code: backend/src/ai-troubleshooting, ai-troubleshooting.service.ts, provider adapter, evidence preprocessor, and frontend/src/pages/ProjectTroubleshooting.jsx.
- How: the service gathers bounded deployment evidence, starts an analysis session, produces a diagnosis, and supports follow-up questions. Deterministic DeployGuard state remains authoritative.
- Live demo: open a failed deployment, start Troubleshooting, show evidence references, ask a follow-up, and show the recommendation.
- Why: AI improves explanation, but must not decide whether a deployment is live, failed, or safe to roll back.
- Failure: provider unavailable, insufficient evidence, unsafe prompt content, or hallucinated advice.
- Status: **PASS** as an advisory feature; live AI-provider availability is **UNVERIFIED**.

Natural answer: “AI explains evidence; it does not control deployment truth.”

14. Alerting & Notification

Claim: important deployment events are sent to users.

- Code: backend/src/notifications, notification-dispatcher.service.ts, sns-notification.adapter.ts, and frontend/src/components/projects/NotificationSettingsPanel.jsx.
- How: preferences control event delivery; notifications are persisted, deduplicated, retried, and passed to a configured provider.
- Live demo: enable notifications, trigger a failed or successful deployment, then show notification history and delivery status.
- Why: users should not need to keep the pipeline page open.
- Failure: provider disabled, invalid destination, retry exhaustion, duplicate event, or rate limit.
- Status: **PASS** locally. External delivery is **UNVERIFIED** unless SNS/email configuration is active.

15. Infrastructure Lifecycle & Demolition

Claim: destroy requires confirmation, cleans AWS, and records the result.

- Code: infrastructure-lifecycle.controller.ts, infrastructure-lifecycle.service.ts, project-deletion.service.ts, and .github/workflows/deployguard-reusable.yml.
- How: confirmation/admission occurs first, Terraform destroys infrastructure, deletion is verified, then non-Terraform cleanup and control-plane finalization occur.
- Live demo: use a disposable project, type the confirmation phrase, run destroy, show Terraform result and final destroyed state.
- Why: destructive operations require an explicit safety boundary and evidence.
- Failure: wrong confirmation, partial AWS deletion, retained secret, stale state, or cleanup failure.
- Status: **PASS** locally. Actual AWS deletion is **UNVERIFIED**.

16. Vendor Independence & Terraform Export

Claim: users can export and retain Terraform code.

- Code: backend/src/terraform-export, terraform-export.controller.ts, TerraformExport.jsx, and verify-terraform-export.ts.
- How: the platform generates an export from canonical runtime configuration rather than arbitrary stale state.
- Live demo: open Infrastructure → Export Terraform, download the archive, inspect variables and modules.
- Why: users retain portability and can review infrastructure outside DeployGuard.
- Failure: incomplete configuration, unauthorized export, export quota, or missing runtime contract.
- Status: **PASS**. Terraform export verification passed.

17. Subscription & Billing

Claim: free and Pro subscriptions, billing status, and plan access exist.

- Code: backend/src/billing, billing-plan.ts, billing.service.ts, billing-provider.service.ts, frontend/src/pages/Billing.jsx, and Stripe integration.
- How: the current code supports free, pro, and pro_plus; subscription state controls entitlements and usage limits. Stripe webhooks update billing state.
- Live demo: open Billing, show current plan, checkout/portal controls, usage limits, and invoice access.
- Why: paid capabilities need server-side entitlement enforcement, not only UI hiding.
- Failure: invalid webhook signature, unpaid subscription, provider outage, quota exceeded, or inconsistent billing state.
- Status: **DIFFERENT FROM PDF**: implementation includes pro_plus, not just Free and Pro. The general billing test passed, but the Stripe contract test currently fails because it expects UI text that is absent or changed. Do not claim Stripe E2E is proven.

Important implementation differences and risks

- resetAndDeployFresh() still dispatches through the normal deployment entry point, but it passes RESET_FRESH; the internal admission/reconciliation path distinguishes it. The low-risk defect test passed: healthy data preservation, five DB states, reset reconciliation, and plan/apply separation.
- Terraform plan failures are classified as DG_TERRAFORM_PLAN_FAILED at terraform_plan; the runtime contract test passed.
- The current Railpack Terraform has ignore_changes = [desired_count, task_definition], protecting direct ECS release ownership.
- MySQL grant reconciliation remains present as DG_MANAGED_MYSQL_GRANT_RECONCILIATION_FAILED.
- No separate presentation deck was provided. I checked the PDF module claims against the implementation. Screenshots in presentation_assets are not proof of current AWS behavior.
- The worktree is dirty, so do not present local source inspection as exact-SHA release certification.

Likely examiner questions

- “What is DeployGuard?”

"It is a deployment control plane that connects GitHub repositories to AWS infrastructure and verifies the running release."

- "Where does the application run?"

  "In ECS Fargate tasks behind an Application Load Balancer, with infrastructure created by Terraform."

- "Why GitHub Actions?"

  "It provides an isolated, asynchronous execution environment for building, scanning, publishing, and deploying."

- "Why Terraform?"

  "Terraform manages declarative infrastructure and state; DeployGuard decides runtime success."

- "What is immutable deployment identity?"

  "The exact source commit, image digest, runtime configuration revision, and task-definition identity."

- "What happens if the deployment fails?"

  "DeployGuard collects stage-specific evidence, classifies the owner and failure code, and exposes safe retry or rollback guidance."

- "Can AI decide whether deployment succeeded?"

  "No. AI is advisory. Deterministic DeployGuard evidence owns lifecycle truth."

- "How is rollback different from redeploy?"

  "Rollback selects an exact previous immutable release; it does not rebuild source."

- "How do you protect database data?"

  "Healthy persistent database identity is preserved, and reset/recovery decisions use reconciliation evidence."

- "How do you prove zero downtime?"

  "I would show two immutable ECS releases, healthy target transitions, and the previous release remaining available during rollout. Source tests alone are not AWS proof."

- "What is not fully proven?"

  "Exact AWS provisioning, live database backup restoration, external notification delivery, live AI provider operation, and Stripe E2E in this checkout."

Five-minute presentation script

"DeployGuard is a deployment orchestration control plane for taking a GitHub application to a verified AWS runtime.

The user first authenticates through GitHub. The backend creates a secure session and applies role-based access control. Important actions are written to an audit log.

The user then creates a project, connects a GitHub repository, selects a branch and deployment directory, and configures environment values. DeployGuard stores this configuration in PostgreSQL.

Before deployment, the repository is inspected using deterministic rules. DeployGuard resolves the application build target, package manager, service directory, and port. It also validates required deployment inputs before making AWS changes.

When the user clicks Deploy, the backend creates an asynchronous operation and dispatches GitHub Actions. The workflow resolves the exact source commit, builds the application with Railpack, validates the image locally, pushes an immutable image to ECR, and optionally scans it with Trivy.

Terraform then creates or updates the required AWS infrastructure: ECS, ALB, networking, secrets, storage, logging, and service discovery. Terraform owns infrastructure existence and state. DeployGuard owns runtime success.

For a managed database, DeployGuard checks database readiness and database-specific evidence before allowing the attached application service to start. Unrelated services should not be blocked only because another service's database is unhealthy.

After ECS starts the application, DeployGuard checks task stability, container exits, target health, and public reachability. Only after these checks pass is the release marked stable.

The platform also provides cost estimates, infrastructure state inspection, monitoring, logs, notifications, AI-assisted troubleshooting, Terraform export, billing, and controlled destruction.

Rollback uses a previously recorded immutable release. It does not rebuild the current source. Destroy requires confirmation, runs Terraform cleanup, verifies AWS deletion, and then finalizes control-plane cleanup.

The most important design decision is separation of responsibilities: Terraform manages infrastructure, ECS runs containers, and DeployGuard decides whether the runtime is successful. This makes failures explainable and prevents infrastructure state from being confused with application health."

Module-by-module demo checklist

1. Login → protected route → role denial → audit log.
2. Create project → GitHub repository → branch → directory → workspace history.
3. Show resolved runtime, service directory, and port.
4. Submit missing/invalid configuration and show pre-flight blocking.
5. Deploy → operation ID → asynchronous stages → terminal result.
6. Show Trivy evidence and advisory/enforced policy behavior.
7. Generate cost estimate → breakdown → approval or warning.
8. Show Terraform events, ECS service, ALB, endpoint, and service discovery.
9. Show Terraform state version, lock, checksum, and validation.
10. Enable EFS/storage → backup status → restore request.
11. Deploy release A → release B → rollback to exact release A.
12. Show pipeline status, CloudWatch logs, runtime metrics, and Grafana.
13. Open troubleshooting on a failure → evidence → diagnosis → follow-up.
14. Enable notifications → trigger event → show delivery status.
15. Destroy disposable project with confirmation → verify final state.
16. Export Terraform → download and inspect archive.
17. Show plan, usage, entitlement, checkout/portal, and invoice state.

Phase

Inspection and viva audit.

Scope

PDF module claims versus current checked-out DeployGuard implementation.

Files Changed

None.

Contract Before

Not assessed before editing; no edits were made.

Contract After

No repository contract changed. Audit findings only.

Test Results

- Authentication verification: **PASS**
- Build-target resolution: **PASS**
- Deployment requirements: **PASS**
- Low-risk defect closure: **PASS**
- Railpack runtime contract: **PASS**
- Trivy controls: **PASS**
- Direct ECS redeploy: **PASS**
- Direct ECS rollback: **PASS**
- Notifications: **PASS**
- Infrastructure lifecycle: **PASS**
- Terraform export: **PASS**
- Billing subscription logic: **PASS**
- Stripe contract: **UNVERIFIED**; current assertion mismatch
- AWS observability contract: **UNVERIFIED**; current assertion mismatch
- PostgreSQL backup/restore execution: **UNVERIFIED**; required environment variable missing
- Full failure-ownership/product tests: **UNVERIFIED**; stale test paths reference missing frontend files

Aws E2E Status

UNVERIFIED. No new exact-SHA AWS certification was executed.

Unverified

Exact AWS deployment, live ECR image, live ECS stability, database restore, MySQL dynamic task-IP proof, external notifications, live AI provider, public endpoint, and Stripe E2E.

Next Bounded Step

Use a clean, exact test1 or explicitly approved test2 commit, repair the stale certification expectations, then run one bounded AWS certification covering deployment, database readiness, runtime health, endpoint verification, evidence persistence, and destroy.